Refinement versus Refresh: Deployment-Time Compute Allocation for Chunked Robot Policies

Alba María Téllez Fernández
MAAT Institute
alba.tellez@maatinstitute.com

Abstract

After training, a chunked robot policy can still be adapted through its inference configuration: how much computation is spent generating one action chunk (refinement), how many actions the chunk contains, and how many are executed before replanning from a new observation (refresh). We formulate this deployment-time allocation problem and evaluate it without updating model weights, using one frozen SmolVLA checkpoint in synchronous LIBERO simulation. Denoising count is the refinement control and chunk length is fixed at $K = 50$. We report closed-loop success alongside dense neural arithmetic and host time under explicit measurement boundaries. Three denoising comparisons were selected on discovery tasks before 1,120 held-out attempts; execution-prefix comparisons were added later and are amended descriptive. Arithmetic and latency characterize per-call cost differently: at $d = 10$, 82.1% of counted arithmetic is independent of denoising, whereas a descriptive affine fit to fixed-input host time assigns 17.7% to its intercept. At $h = 2$, $d = 2$ succeeds on 139/160 attempts versus 122/160 for $d = 5$ ($+10.6$ percentage points; task-conditional 95% interval: $+4.4$ to $+16.9$), while using 5.9% less arithmetic per control step. Executing $h = 5$ rather than $h = 2$ at $d = 2$ yields 138/160 versus 139/160 successes and 60.7% lower arithmetic per attempt; equivalence is not established. These checkpoint-specific results illustrate an evaluation procedure for post-training deployment adaptation; they do not define a universal setting.

1 Introduction

Post-training adaptation need not modify a robot policy's weights. A frozen chunked policy can also be adapted through its deployment configuration: how much computation is spent generating one action chunk, how many actions the chunk contains, and how many are executed before the policy replans from a new observation. These controls change both deployment cost and closed-loop behavior. We call adjustable computation within one policy call *refinement*. We call obtaining a new observation-conditioned prediction *refresh*; the execution prefix controls how often it occurs. Predicted chunk length determines how many actions each call constructs.

These choices couple computation and feedback. Additional refinement can alter the predicted actions. A longer execution prefix amortizes a policy call, but it delays feedback and exposes later actions from an older prediction to the environment. We therefore ask one allocation question: *under an explicit compute ceiling, which refinement, chunk-length, and refresh settings give the best closed-loop behavior?* We do not trade a fixed budget continuously between refinement and refresh; we compare configurations that are feasible under common compute ceilings and report their nominal rates and realized attempt costs.

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: Post-Training Adaptation of Robot Foundation Models.

We study this form of post-training adaptation as a controlled allocation problem. Our evaluation procedure separates a model-specific refinement configuration, predicted chunk length, and execution prefix. It measures every policy call under a stated boundary, aggregates cost over complete attempts, uses discovery tasks to choose prespecified comparisons, and evaluates those comparisons on separate held-out tasks. Analyses added after outcomes were visible are labelled *amended descriptive*. This procedure supports choosing a deployment configuration; it does not train the policy or learn an online allocator.

The procedure is intended for chunked policies with a controllable refinement setting. Depending on the architecture, that setting could control integration steps, candidate generation, search, or verification effort. This paper instantiates only denoising count, and control values should not be compared across architectures. Synchronous execution keeps sequencing and measurement boundaries fixed; the study does not determine an asynchronous schedule, physical-robot latency, or a hardware-independent optimum.

We use one frozen SmolVLA checkpoint as a worked case study [1]. Predicted chunk length remains fixed at $K = 50$ and is not evaluated. We test 12 configurations in 480 discovery attempts, then seven settings in 1,120 held-out attempts on eight tasks excluded from selection. Refinement comparisons were prespecified; the available execution-prefix comparisons were added later and remain descriptive.

The contributions are:

- a formulation connecting a model-specific refinement configuration, predicted chunk length, and execution prefix to nominal compute rate and realized attempt cost;
- an evaluation procedure for post-training deployment adaptation that separates discovery, prespecified held-out comparisons, and amended descriptive analyses; and
- a SmolVLA case study showing that a large per-call cost can be independent of the refinement control, that lower refinement improves held-out success in the primary comparison, and that quality preservation under less frequent refresh remains unresolved.

2 Related Work

Post-training allocation under a compute budget. Compute-optimal scaling studies ask how a fixed training budget should be divided between model size and data [2]; related embodied work studies scaling in agent pretraining and world models [3]. Including deployment demand can change the preferred allocation [4]. These studies establish the need to state the budget and counting boundary. We instead adapt the inference configuration of one frozen policy and evaluate each deployment-time compute allocation by closed-loop task success.

Computation used to generate an action. Diffusion Policy generates action sequences through iterative conditional denoising [5]. RoboMonkey spends additional inference compute on action candidates and verification [6]. This is one form of refinement: both proposal and verification cost belong to the policy call. SmolVLA provides the flow-matching policy used in our case study and also studies asynchronous inference [1]. VLA-Perf examines how model architecture, hardware, placement, networking, and asynchronous execution affect VLA inference performance [7]. These works show that arithmetic and elapsed time depend on both the model and its execution system. Our case study holds the checkpoint, hardware, and synchronous execution path fixed so that the effects of refinement and execution prefix can be compared within one system.

Choosing when to refresh. ACT predicts action chunks to reduce the effective decision horizon [8]; chunked action spaces have also been studied for reinforcement learning [9]. Adaptive Action Chunking uses action entropy to choose how much of a predicted chunk to execute [10]. DVAC uses variation observed during denoising to adapt the execution prefix [11]. ElegantVLA schedules vision–language recomputation, reuse, and action generation during execution [12]. These methods change the schedule within an episode. We map fixed refinement and prefix settings first, separate selection from held-out evaluation, and report both nominal compute rate and realized attempt cost. This design evaluates axes available to an adaptive method; it does not compare a fixed deployment configuration with an online, within-episode schedule.

3 Allocating Deployment-Time Compute Between Refinement and Refresh

We represent the post-training deployment configuration of a frozen chunked policy by its model-specific refinement configuration $\mathbf{r}$, predicted chunk length K, and execution prefix h. The following quantities connect those controls to the two cost measures used in the evaluation.

3.1 Nominal Rate and Realized Attempt Cost

At each policy query, $1 \leq h \leq K$ actions are executed and the remaining $K - h$ are discarded before replanning. For fixed or reference-calibrated call cost, let $F_{\text{gen}}(\mathbf{r}, K)$ be measured arithmetic under a stated boundary. The nominal compute rate per simulated control step is

$$b(h, K, \mathbf{r}) = \frac{F_{\text{gen}}(\mathbf{r}, K)}{h}.$$

Let $p_{\text{success}}(h, K, \mathbf{r})$ denote expected closed-loop success under the specified task distribution. For a candidate set $\mathcal{C}$ and budget ceiling B, the allocation target is

$$(h^\star, K^\star, \mathbf{r}^\star)(B) \in \arg \max_{(h,K,\mathbf{r}) \in \mathcal{C}:\, b(h,K,\mathbf{r}) \leq B} p_{\text{success}}(h, K, \mathbf{r}). \tag{1}$$

A configuration need not exhaust the ceiling. Equation 1 defines a within-checkpoint deployment decision; paired rollouts estimate its unknown success term.

In the SmolVLA instantiation, $K = 50$ and $\mathbf{r} = (d)$, where d is the denoising count. Figure 1 shows this case-study execution loop. For compactness below, $F_{\text{gen}}(d)$ denotes $F_{\text{gen}}((d), 50)$ and $b(h, d)$ denotes $b(h, 50, (d))$.

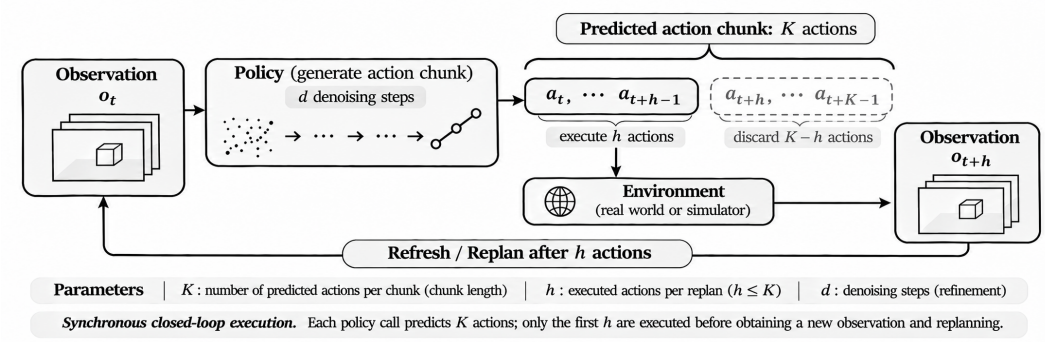


Figure 1: Closed-loop execution in the SmolVLA case study. Each policy call predicts K actions after d denoising steps, executes the first h, discards the remainder, and replans from a new observation. Here $\mathbf{r} = (d)$ and $K = 50$.

We distinguish this rate from total arithmetic per attempt. If attempt e executes T_e actions, let $N_e = \lceil T_e/h \rceil$ be its number of policy calls and let $F_{e,j}(\mathbf{r}, K)$ be the measured cost of call j. Attempt e's generated-chunk expenditure is

$$C_e = \sum_{j=1}^{N_e} F_{e,j}(\mathbf{r}, K). \tag{2}$$

The final generated chunk is charged even if only part of its execution prefix is used. For the case study, each call has fixed cost for a given d, so this reduces to $C_e = \lceil T_e/h \rceil F_{\text{gen}}(d)$. The summed form also covers variable-cost early exit, sampling, search, verification, or language generation. All-attempt means include unsuccessful episodes. Different trajectories and termination times can therefore change expenditure at the same nominal rate. FLOPs, host elapsed time, and CUDA-stream time retain separate units and measurement boundaries.

4 SmolVLA Case Study

This section applies the allocation procedure to adapt the inference configuration of one frozen checkpoint. Discovery maps the candidate settings and freezes the comparisons; confirmation then tests those comparisons on tasks excluded from selection.

4.1 Discovery and confirmation design

We call evaluation on tasks excluded from selection the *held-out confirmation phase*, or confirmation for short.

We evaluate a public SmolVLA checkpoint on LIBERO [13]. The predicted chunk length is fixed at $K = 50$, and the model's native deployment uses $(h, d) = (1, 10)$. We hold the checkpoint, observations, preprocessing, action normalization, control period, and task-specific episode horizons fixed.

The discovery study varies $h \in \{1, 2, 5\}$ and $d \in \{2, 5, 10, 20\}$ on four Spatial tasks. Each task has five initial states and two rollout seeds, giving 40 paired task–state–seed identities. Every identity receives all twelve settings in randomized order, for 480 attempts. Two discovery tasks overlap pre-study pilot work; the other two were chosen before their study outcomes were observed. This partial prior exposure is part of the reason that discovery is used for selection rather than final evaluation.

After discovery, we freeze three denoising comparisons, one at each compute ceiling. Their selected settings and comparators, plus native deployment, produce seven distinct settings. Confirmation evaluates all seven on eight Goal and Object tasks excluded from selection. Each task has ten initial states and two rollout seeds, giving 160 attempts per setting and 1,120 in total. Prior exposure to these tasks during unrelated development is not assessed. No equivalence margin was prespecified, so similar observed success cannot establish equivalent quality. Appendix C.4 records which comparisons were frozen and which were added later as descriptive analyses. The settings, identities, stopping rules, inclusion criteria, and primary analysis were fixed before confirmation outcomes. During collection, after 114 attempts per primary setting, an execution-order amendment changed only the remaining order; Appendix C.4 gives the phase history.

Conditions are paired by initial state and rollout seed. They also share policy noise while their queries occur at the same simulation step. Every denoising setting traverses the same integration interval; changing d changes only its discretization. Internal checks cover initial conditions, generated and executed actions, and recorded outcomes. They do not provide an independent simulator replay; Appendix C.3 states the validation boundary.

Success gives equal weight to each task; because every task has the same number of states and seeds, this equals the overall success fraction. Descriptive 95% intervals use 20,000 bootstrap draws of states within the fixed tasks while retaining paired seeds and settings. The intervals condition on the four discovery tasks or eight confirmation tasks, and secondary intervals are not adjusted for multiple comparisons. Arithmetic includes every generated chunk, including failed attempts and partly consumed final prefixes. Appendix A.3 reports timing separately.

4.2 A fixed-cost floor in the SmolVLA implementation

For the case-study checkpoint and reference input, generation cost separates into a denoising-independent term and a term that grows linearly with d. To rounding, the measured arithmetic is

$$F_{\text{gen}}(d) \simeq 527.108 + 11.457d \quad \text{GFLOPs}$$

at all six calibrated denoising settings. Native generation at $d = 10$ costs 641.675 GFLOPs, of which 82.1% is independent of d. Halving denoising from ten to five saves 8.9% of counted generation work; reducing it to two saves 14.3%. Equivalently, relative to native deployment $(h, d) = (1, 10)$,

$$\frac{b(h, d)}{b_{\text{native}}} \simeq \frac{0.821 + 0.179(d/10)}{h}.$$

Here $b_{\text{native}} = b(1, 10)$.

Profiling counts matrix products, convolutions, and attention across vision, language prefill, and the action expert, using two FLOPs per multiply-add. Embedding lookups, normalization, activations, elementwise arithmetic, and external preprocessing are excluded. We therefore refer to *dense neural arithmetic*, rather than every scalar operation. Calibration uses one real development observation and fixed tensor shapes; Appendix A.2 gives the procedure.

We prespecify compute-rate ceilings of 0.25, 0.50, and 1.00 times native. Eligibility uses measured per-d values, not the rounded affine expression. For example, $(1, 2)$ costs 0.857 times native rate and is ineligible at half-native budget, whereas $(2, 2)$ costs 0.429 times native. The denoising-independent work alone excludes $h = 1$ at half-native budget and $h \leq 3$ at quarter-native budget. A tighter budget can therefore force a longer execution prefix before any quality-based choice is made. This is an implementation-specific arithmetic floor under the declared counting boundary, not a universal VLA scaling law.

Calibration uses a fixed input shape. A separate audit using the language lengths observed in the tasks leaves eligibility unchanged under the same absolute ceilings (Appendix A.5). The reference-input arithmetic used for selection remains distinct from task-specific arithmetic and elapsed time. A descriptive affine fit to the six fixed-input host-time means gives $T_{\text{gen}}(d) \simeq 112.3 + 52.2d$ ms. Its intercept is 17.7% of fitted $d = 10$ host time, compared with 82.1% for counted arithmetic. The timing intercept is an empirical fit under one host boundary, not a component attribution. Consistently, reducing d from 10 to 5 lowers measured host time by 40.5% while counted arithmetic falls by 8.9% (Appendix A.3); FLOPs are therefore not a latency proxy. The cost model determines which settings satisfy a ceiling. The following experiments test their closed-loop outcomes.

4.3 Discovery: observed success is not monotonic in refinement

Native $(1, 10)$ succeeds in 25/40 discovery attempts. The highest observed count is 28/40 for $(1, 5)$; $(1, 2)$ and $(2, 2)$ each succeed in 26/40. At $h = 5$, success falls from 21/40 to 14/40 as d increases from 2 to 20, with intermediate counts of 18/40 and 16/40. Appendix Figure 3 shows every configuration. These point estimates do not justify imposing monotonic improvement with denoising. The small paired sample also does not certify equal quality for configurations with similar observed counts.

Selection also depends on how cost is defined. Nominal arithmetic rate differs from realized attempt expenditure, and arithmetic savings differ from elapsed time. Replacing the arithmetic ceiling with a host-time ceiling changes which prefixes are feasible. This sensitivity analysis does not alter the frozen arithmetic-based experiment. Appendix A.1 gives the full discovery results, and Appendix A.4 gives the resource-coordinate analysis.

4.4 Selection rules and prespecified comparisons

Discovery freezes $(5, 2)$, $(2, 2)$, and $(1, 2)$ at the quarter-, half-, and full-native ceilings, with $(5, 5)$, $(2, 5)$, and $(1, 5)$ as their respective comparators; $(2, 2)$ versus $(2, 5)$ is primary. Every selected setting is an observed grid point, so confirmation tests transfer to new tasks rather than extrapolation. Each pair varies denoising at a fixed prefix and compares two feasible settings; the pair does not divide an identical compute rate.

At the primary ceiling, $(2, 2)$ has the highest eligible discovery success. Among the remaining settings, $(2, 5)$ ties $(2, 10)$ and wins the lower-cost tie-break. A discovery-fitted logistic model and a simpler rule—fewest denoising steps, then shortest feasible prefix—produce the same three selections. Because the simpler rule was documented after data collection began, the agreement is descriptive and does not show an advantage from fitting. Appendix B gives the model, candidate set, and diagnostics.

5 Held-Out Success and Compute

The held-out evaluation applies seven settings to the same 160 task–state–seed identities, for 1,120 planned attempts. Table 1 reports success and both compute measures; Figure 2 places them in one view and shows the paired comparisons. Three fixed-h comparisons of $d = 2$ against $d = 5$ were prespecified, with $(2, 2)$ versus $(2, 5)$ as primary. Prefix comparisons were added after outcomes were visible and remain descriptive. Appendix C.3 documents two removed duplicate records without changing the planned totals.

Table 1: Held-out results for the seven SmolVLA settings. Each setting has 160 attempts across eight tasks. The predicted chunk length is fixed at $K = 50$. Nominal rate is counted arithmetic per control step relative to native $(1, 10)$; mean arithmetic per attempt charges every generated chunk.

Allocation		Success		Nominal	Mean per attempt	
h	d	Count	(%)	rate ratio	TFLOPs	Steps
1	2	137/160	85.6	0.857	83.3	151.4
1	5	136/160	85.0	0.911	89.1	152.5
1	10	135/160	84.4	1.000	98.4	153.4
2	2	139/160	86.9	0.429	41.1	149.0
2	5	122/160	76.3	0.455	48.7	166.3
5	2	138/160	86.3	0.171	16.1	144.9
5	5	133/160	83.1	0.182	18.5	156.2

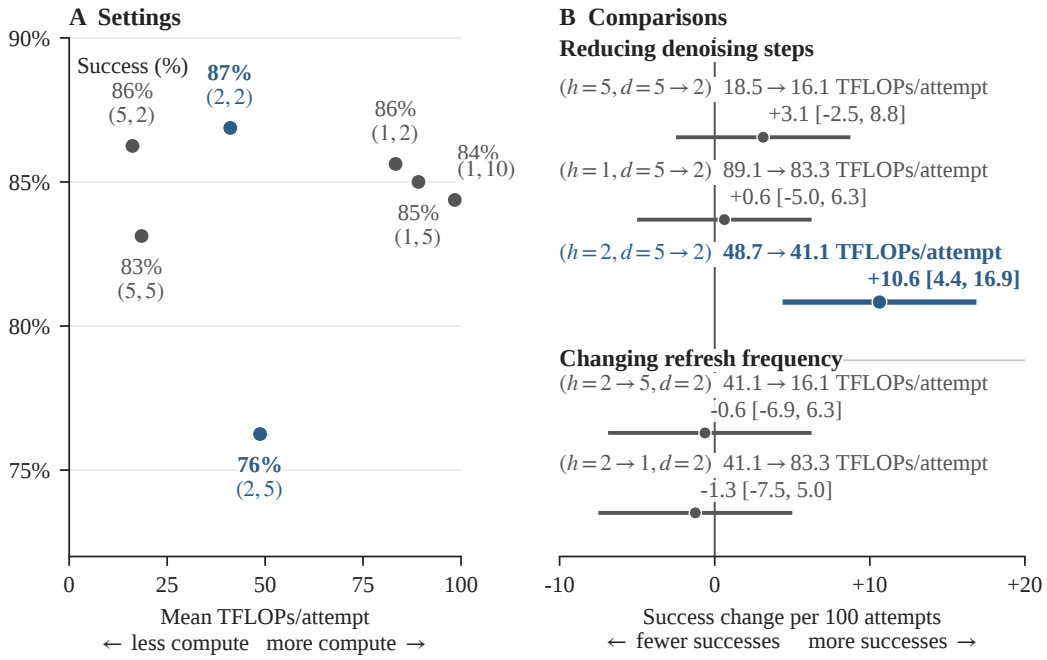


Figure 2: Success and compute in the held-out evaluation. Panel A shows the seven settings (h: executed prefix; d: denoising steps). Horizontal position is mean reference-calibrated arithmetic per attempt, including failures; the success axis begins at 72%. Blue marks the primary pair. Panel B reports the paired success change for each displayed move and its task-conditional 95% interval. A positive value favors the destination setting, and an interval crossing zero does not separate the settings under this analysis. The $h = 2 \rightarrow 5$ row therefore reverses the sign used for $(2, 2) - (5, 2)$ in Table 9. Arithmetic per attempt is shown beneath each comparison. Selection ceilings instead use nominal arithmetic per control step (Section 4.2). Exact counts and two additional descriptive contrasts appear in Tables 1 and 9.

Primary result: lower refinement improves success while reducing compute. The primary comparison holds $h = 2$ fixed and changes only denoising. Moving from $d = 5$ to $d = 2$ raises success from 122/160 to 139/160, a paired increase of 10.6 percentage points with a task-conditional 95% interval of $[+4.4, +16.9]$. Counted arithmetic also decreases: by 5.9% per control step and 15.6% per attempt, from 48.7 to 41.1 TFLOPs. Thus, for this checkpoint at $h = 2$, the additional denoising is both less successful and more expensive.

The primary effect is consistent across tasks, but not across prefixes. The result is not driven by one task: seven tasks favor $d = 2$ and one ties, and the paired identities contain 24 wins and seven losses. Nevertheless, it is not a general denoising trend. At $h = 1$ and $h = 5$, the point estimates also favor $d = 2$, but their intervals cross zero. The response model predicted the primary direction but understated its magnitude ($+4.0$ predicted versus $+10.6$ observed), so these data do not demonstrate calibrated forecasting. Appendix C.2 reports task sensitivity, trajectory lengths, and expenditure intervals. Counted arithmetic should not be interpreted as physical latency.

Longer execution prefixes save substantially more compute, with quality unresolved. The descriptive prefix comparisons ask whether acting longer before replanning can reduce cost without an obvious loss in observed success. At $d = 2$, executing five rather than two actions per prediction reduces mean arithmetic per attempt by 60.7%, from 41.1 to 16.1 TFLOPs, while the observed totals differ by one success (138/160 versus 139/160). The uncertainty interval spans practically meaningful differences in both directions, and task-level effects are mixed. The result therefore shows a large compute saving, but it does not establish equivalent quality.

More frequent refresh shows no measured success advantage in these contrasts. Refreshing after every action is still more expensive and shows no measured success advantage over refreshing after two actions. Taken together, the prefix comparisons show why cost and quality must be reported separately: the cost ranking is clear, whereas the quality ranking is not. Because these comparisons were defined after outcomes were visible, they motivate a future prespecified study rather than support a general recommendation about refresh frequency. They also cannot reconcile the different discovery and confirmation patterns, which were measured on different task sets.

Appendix C.1 records the comparison provenance and the two contrasts involving native $(1, 10)$; Table 8 shows task-level variation.

Evidence boundary. The prespecified evidence establishes the primary result for this checkpoint at $h = 2$; it does not establish a uniform refinement effect. The later prefix comparisons establish their arithmetic differences under the declared counting boundary, while their success differences remain descriptive and uncertain.

Scope and measurement limits. Neither varied control isolates a single mechanism: changing denoising changes numerical integration, while changing the prefix changes both observation age and executed action positions. The study uses one checkpoint, twelve tasks, and synchronous simulation; its intervals are conditional on the sampled tasks, and no equivalence margin was prespecified. Counted arithmetic excludes some operation classes and is not a latency proxy. Recorded outcomes pass internal source and consistency checks, but the full confirmation trajectories were not replayed in an independent, version-matched simulator. Appendix C.3 defines this validation boundary.

6 Discussion and Conclusion

Adapting a frozen policy through its deployment configuration can change both success and total compute. In the primary SmolVLA comparison, reducing denoising from five to two steps improves held-out success while reducing counted arithmetic by 5.9% per control step and 15.6% per attempt. The other prespecified prefixes do not establish a uniform refinement effect. Executing five rather than two actions before refresh produces the much larger compute saving—60.7% per attempt—but the available descriptive comparison does not establish equivalent success. These results show why success, nominal rate, and complete attempt cost must be evaluated together.

The transferable contribution is an evaluation procedure for post-training deployment adaptation, rather than a universal setting. Within each frozen model, define its refinement configuration and

predicted chunk length, measure every policy-call cost under an explicit boundary, vary the execution prefix, freeze comparisons before held-out evaluation, and report realized attempt cost with success. A refinement configuration can include integration steps, autoregressive reasoning, candidate sampling, search, verification, or cache reuse, provided each component's cost is measured. When comparing architectures, the meaningful evidence is each model's within-model allocation result. Control values and counting boundaries that differ across models should not be pooled directly.

The present study keeps $K = 50$ throughout, leaving predicted chunk length as a central untested allocation lever; with the tested prefixes, 90–98% of every prediction is discarded before the next query. The next study should prespecify a quality-preservation test of the cheaper $(5, 2)$ setting, vary K, and separate observation freshness from action-position effects. Repeating the same within-model design across checkpoints and before and after weight-based post-training adaptation would determine whether those adaptations also change the preferred deployment configuration. This is the evidence needed to decide when compute should refine the current prediction and when it should produce a new prediction from fresher feedback.

Data and Code Availability

Maintained evaluation infrastructure, the camera-ready manuscript, and compact confirmation records sufficient to reproduce the held-out totals and paired primary comparison are available in the BenchStudio worked example at `https://github.com/maatinstitute/BenchStudio/tree/main/examples/refinement_refresh`. The example also records cryptographic identities for the retained historical archive. Action tensors, images, checkpoint weights, and simulator assets are not redistributed.

References

[1] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Remi Cadene. SmolVLA: A vision-language-action model for affordable and efficient robotics, 2025. URL `https://arxiv.org/abs/2506.01844v1`. arXiv:2506.01844v1.

[2] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Thomas Hennigan, Eric Noland, Katherine Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karén Simonyan, Erich Elsen, Oriol Vinyals, Jack Rae, and Laurent Sifre. An empirical analysis of compute-optimal large language model training. In *Advances in Neural Information Processing Systems*, volume 35, 2022. doi: 10.52202/068431-2176. URL `https://proceedings.neurips.cc/paper_files/paper/2022/hash/c1e2faff6f588870935f114ebe04a3e5-Abstract.html`.

[3] Tim Pearce, Tabish Rashid, David Bignell, Raluca Georgescu, Sam Devlin, and Katja Hofmann. Scaling laws for pre-training agents and world models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48542–48562. PMLR, 2025. URL `https://proceedings.mlr.press/v267/pearce25a.html`.

[4] Nikhil Sardana, Jacob Portes, Sasha Doubov, and Jonathan Frankle. Beyond Chinchilla-Optimal: Accounting for inference in language model scaling laws. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 43445–43460. PMLR, 2024. URL `https://proceedings.mlr.press/v235/sardana24a.html`.

[5] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin C. M. Burchfiel, and Shuran Song. Diffusion Policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, July 2023. doi: 10.15607/RSS.2023.XIX.026. URL `https://roboticsproceedings.org/rss19/p026.html`.

[6] Jacky Kwok, Christopher Agia, Rohan Sinha, Matt Foutter, Shulu Li, Ion Stoica, Azalia Mirhoseini, and Marco Pavone. RoboMonkey: Scaling test-time sampling and verification for vision-language-action models. In *Proceedings of the 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 3200–3217. PMLR, 2025. URL https://proceedings.mlr.press/v305/kwok25a.html.

[7] Wenqi Jiang, Jason Clemons, Karu Sankaralingam, and Christos Kozyrakis. How fast can I run my VLA? Demystifying VLA inference performance with VLA-Perf, 2026. URL https://arxiv.org/abs/2602.18397v1. arXiv:2602.18397v1.

[8] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, July 2023. doi: 10.15607/RSS.2023.XIX.016. URL https://roboticsproceedings.org/rss19/p016.html.

[9] Qiyang Li, Zhiyuan (Paul) Zhou, and Sergey Levine. Reinforcement learning with action chunking. In *Advances in Neural Information Processing Systems*, volume 38, 2025. doi: 10.52202/085713-1856. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/50348e8f9aef984abe0ea1ec2a326f78-Abstract-Conference.html.

[10] Yuanchang Liang, Xiaobo Wang, Kai Wang, Shuo Wang, Xiaojiang Peng, Haoyu Chen, David Kim Huat Chua, and Prahlad Vadakkepat. Adaptive action chunking at inference-time for vision-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 20802–20811, June 2026. URL https://openaccess.thecvf.com/content/CVPR2026/papers/Liang_Adaptive_Action_Chunking_at_Inference-time_for_Vision-Language-Action_Models_CVPR_2026_paper.pdf.

[11] Xiangdong Feng, Yuxuan Cheng, Chen Shi, Boyao Han, Yuxuan Yan, Yitong Hong, Zhuotao Tian, and Li Jiang. Denoising tells when to replan: Denoising-variance adaptive chunking for flow-based robot policies, 2026. URL https://arxiv.org/abs/2606.03847v1. arXiv:2606.03847v1.

[12] Ye Li, Huanan Liu, Kangye Ji, Yuan Meng, Jiajun Fan, Yuansong Wang, Shiyu Qin, Chenglei Wu, Shu-Tao Xia, and Zhi Wang. ElegantVLA: Learning when to think for efficient vision-language-action models, 2026. URL https://arxiv.org/abs/2605.29438v1. arXiv:2605.29438v1.

[13] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. doi: 10.52202/075280-1939. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/8c3c666820ea055a77726d66fc7d447f-Abstract-Datasets_and_Benchmarks.html.

A Discovery and Resource Measurements

Appendices A–C follow the study chronology. This section reports discovery outcomes, generation-cost calibration, fixed-input timing, sensitivity to the resource coordinate, and the language-shape audit. Appendix B then documents frozen selection, and Appendix C reports held-out outcomes and verification.

A.1 Discovery outcomes

The checkpoint is `HuggingFaceVLA/smolvla_libero`, revision `6721902bc4d61e50a3bfdb11dfb4cb626f05d102`. Predicted chunk length is 50; native execution uses $h = 1, d = 10$. Discovery uses Spatial tasks 5–8, initial states 10–14, and rollout seeds 2000 and 2001. The 20 task–state blocks contain both seeds and all 12 configurations. Discovery bootstrap resampling uses 20,000 draws with analysis seed 20260913.

Table 2: Discovery successes on Spatial tasks 5–8. Each task contributes five states and two seeds per configuration. The nominal rate ratio is $F_{\text{gen}}(d)/(hF_{\text{gen}}(10))$, not realized attempt cost. The best observed count is 28/40 at $(1, 5)$; no monotonic denoising trend is assumed.

Allocation		Spatial task successes (each /10)					Nominal
h	d	Task 5	Task 6	Task 7	Task 8	Total	rate ratio
1	2	5	7	4	10	26/40	0.857
1	5	5	8	6	9	28/40	0.911
1	10	5	7	6	7	25/40	1.000
1	20	5	6	6	8	25/40	1.179
2	2	5	7	4	10	26/40	0.429
2	5	4	6	4	9	23/40	0.455
2	10	4	6	4	9	23/40	0.500
2	20	4	7	4	9	24/40	0.589
5	2	6	5	4	6	21/40	0.171
5	5	2	5	5	6	18/40	0.182
5	10	2	4	5	5	16/40	0.200
5	20	2	3	4	5	14/40	0.236

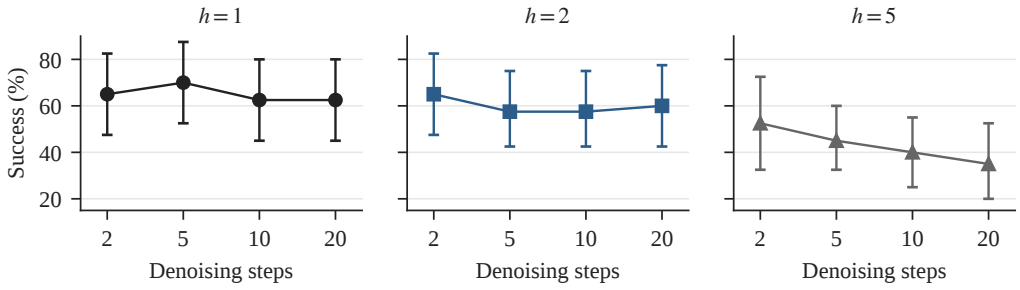


Figure 3: Discovery success by executed prefix h, with 40 attempts per configuration on four Spatial tasks. Connected estimates make the within-prefix response to denoising visible; whiskers are descriptive, task-conditional 95% paired state-bootstrap intervals. The d axes are categorical.

A.2 Generation calibration and discovery resources

Calibration uses one real observation from Spatial task 0, initial state 0, and common explicit noise. For each $d \in \{2, 3, 5, 8, 10, 20\}$, profiling records solver timesteps and dense operation counts, separately from elapsed timing. Each setting receives 20 warm-up generations, followed by three

randomized timing blocks of 20 measured calls per setting. The rollout evaluator also performs 20 warm-up generations once per configuration.

Host generation timing brackets prediction and completion synchronization; CUDA events measure the current-stream prediction interval. The outer rollout clock additionally includes observation processing, action handling, simulation/rendering, and record writing. Model loading, environment creation, reset, and warm-up are measured separately. Process sampling occurs approximately every five seconds; the largest recorded gap during discovery is below 5.03 seconds. Display processes remain on the measurement GPU.

Table 3: Discovery resource measurements. Generated-chunk totals use the declared dense-arithmetic boundary and charge partially executed final prefixes. Episode means include unsuccessful attempts. $(2, 2)$ has 26/40 observed successes at 42.9% of the native nominal arithmetic rate.

Allocation		Nominal / step	Mean per attempt		Success
h	d	GFLOPs	TFLOPs	Rollout (s)	Count
1	2	550.0	90.6	39.5	26/40
1	5	584.4	91.7	61.8	28/40
1	10	641.7	109.6	111.9	25/40
1	20	756.2	129.1	199.8	25/40
2	2	275.0	45.0	21.8	26/40
2	5	292.2	53.4	38.5	23/40
2	10	320.8	57.8	61.6	23/40
2	20	378.1	67.8	107.8	24/40
5	2	110.0	20.5	12.7	21/40
5	5	116.9	23.8	20.2	18/40
5	10	128.3	27.2	32.0	16/40
5	20	151.2	33.1	55.6	14/40

Calibration requires approximately 291 seconds and discovery 31,245 seconds (8.68 hours) of host elapsed time, including setup and bookkeeping. This search expenditure is additional to subsequent deployments. The experiment does not estimate an amortization threshold or a training–deployment cost crossover.

All measurements use an RTX 4060 Ti. Process samples taken approximately every five seconds show no unrelated sampled CUDA compute during calibration, discovery, or the initially scheduled confirmation phase; display processes remain. Monitoring gaps in later phases make aggregate confirmation rollout times descriptive. Host and CUDA-stream generation intervals are measured separately from the outer synchronous rollout interval, and warm-up is excluded.

A.3 Fixed-input generation timing

Table 4: Fixed-input full-generation calibration. Each setting has 60 calls in three randomized blocks. Range gives the minimum and maximum host block means, not a confidence interval. Host time falls more steeply with d than counted arithmetic. Host and CUDA columns average separate retained measurements of all 60 calls; the extra decimals distinguish their means, not timing accuracy.

d	GFLOPs	Host (ms)	CUDA (ms)	Host block range (ms)
2	550.0	217.891	217.862	214.5–220.1
3	561.5	269.990	269.970	265.3–272.6
5	584.4	374.321	374.300	366.8–378.1
8	618.8	529.746	529.726	519.9–536.1
10	641.7	629.624	629.604	623.2–642.2
20	756.2	1158.635	1158.608	1133.5–1172.6

The calibration stores full-generation intervals and global operator arithmetic counts, not separate vision, language-prefill, or action-expert timings and arithmetic totals. These records cannot attribute elapsed sensitivity to a particular component, memory traffic, omitted operation classes, synchroniza-

tion, or software overhead. CUDA-event intervals are elapsed stream measurements, not summed active-kernel durations. No component-level decomposition is inferred.

A.4 Resource-coordinate sensitivity

Arithmetic savings and elapsed savings differ. Mean rollout time is 111.9 seconds per attempt for native deployment, 61.8 seconds for $(1, 5)$, and 21.8 seconds for $(2, 2)$. Their mean generated-chunk expenditures are 109.6, 91.7, and 45.0 TFLOPs, respectively. These are observed all-attempt means under the same task allocation; they include different trajectories and outcomes. Table 3 reports all discovery configurations.

There is also an ordering reversal: $(1, 5)$ uses more arithmetic per attempt than $(2, 20)$ (91.7 versus 67.8 TFLOPs), but less rollout time (61.8 versus 107.8 seconds). These means include different trajectories. On a fixed input, reducing d from 10 to 5 lowers mean generation host time from 629.6 to 374.3 ms (40.5%), despite only 8.9% less counted arithmetic. The calibration therefore also separates arithmetic savings from elapsed savings without trajectory differences. It does not resolve their component-level cause. A FLOP-constrained choice is not necessarily a latency- or throughput-optimal choice.

As a descriptive sensitivity analysis, define calibrated host-time rate $b_T(h, d) = \overline{T}_{\mathrm{gen}}(d)/h$, the amortized fixed-input generation time per simulated control step, and normalize it by native host-time rate. For $(1, 2)$, $(2, 2)$, and $(5, 2)$, the normalized arithmetic rates are 0.857, 0.429, and 0.171, while the normalized host-time rates are 0.346, 0.173, and 0.069. Consequently, $(1, 2)$ is ineligible at a half-native arithmetic ceiling but eligible at a separately normalized half-native host-time ceiling. These resource constraints admit different refresh schedules; the choices do not establish quality optima. The frozen experiment remains arithmetic-based.

A.5 Language-shape transport audit

The profiled batch contains one sample, two $3 \times 360 \times 360$ image tensors, an eight-dimensional state tensor, and language-token and attention-mask tensors of length 20. The checkpoint exposes seven-dimensional actions and generates a 50-action chunk. Evaluation fixes batch size, image preprocessing, state/action dimensions, and predicted chunk length. The tokenizer pads language only to the longest sequence in a batch. Reconstructing the declared task strings with the pinned tokenizer gives 15–17 tokens for discovery and 5–12 for confirmation, versus 20 in calibration. We profile every distinct evaluated length for $d \in \{2, 5, 10\}$, preserving all other shapes, and test eligibility against the prespecified absolute ceilings. Table 5 reports the range over evaluated lengths 5–17 against the original length-20 value.

Table 5: Dense generation arithmetic across evaluated language shapes. Budget tests retain the prespecified absolute ceilings; none of the reported confirmation configurations changes eligibility.

d	Evaluated lengths GFLOPs (range)	Length 20 GFLOPs	Eligibility changes
2	539.7–547.9	550.0	0
5	573.5–582.2	584.4	0
10	629.8–639.3	641.7	0

In particular, $(2, 10)$ remains within the original half-native ceiling for all evaluated lengths; shorter inputs reduce rather than increase its counted work. The audit changes neither policy execution nor the frozen selections.

B Frozen Selection and Diagnostic Analyses

B.1 Frozen selection procedure

We compare two logistic response-model families. A compute-only model uses a quadratic in log normalized rate. An allocation model uses $\log h$, $\log(d/10)$, both squared terms, and their interaction. Each family is fitted with ridge penalty 1 or 10. Leave-one-task-out log loss selects the model, which

is then refitted on the discovery set. No individual seed repeat is held out while another repeat from the same task remains in training.

The candidate set is $h \in \{1, 2, 3, 5\}$ crossed with $d \in \{2, 3, 5, 8, 10, 20\}$: the 12 discovery configurations and 12 untested configurations. Their calibrated costs determine eligibility; the fitted model selects the highest predicted success under each ceiling. Ties favor lower compute, then lexicographic order.

Table 6: Discovery-based selector choices. The learned choices and comparisons were frozen before confirmation; additional selector baselines were documented after collection began. The empirical selector uses the measured grid; other selectors use the declared candidate set. The simple rule exactly matches the learned choices. The selected rate ratio is the learned recommendation's nominal arithmetic rate divided by the native rate. Prespecified comparators and predicted gaps appear in Table 7; the half-native row is primary.

	Selector choice (h, d)				Learned selector
Ceiling ratio	Learned	Simple	Empirical	Compute-only	Rate ratio
0.25	$(5, 2)$	$(5, 2)$	$(5, 2)$	$(5, 20)$	0.171
0.50 (primary)	$(2, 2)$	$(2, 2)$	$(2, 2)$	$(2, 10)$	0.429
1.00	$(1, 2)$	$(1, 2)$	$(1, 5)$	$(1, 10)$	0.857

Table 7: All prespecified denoising comparisons: selected minus comparator, holding h fixed within each row. Predicted and observed gaps and intervals are percentage points. The half-native row is primary; secondary intervals are descriptive, task-conditional, and unadjusted.

	Setting (h, d)		Success difference (pp)		
Ceiling ratio	Selected	Comparator	Predicted	Observed	95% interval
0.25	$(5, 2)$	$(5, 5)$	$+6.5$	$+3.1$	$[-2.5, +8.8]$
0.50 (primary)	$(2, 2)$	$(2, 5)$	$+4.0$	$+10.6$	$[+4.4, +16.9]$
1.00	$(1, 2)$	$(1, 5)$	$+2.2$	$+0.6$	$[-5.0, +6.3]$

B.2 Selector diagnostics

The simple rule defined in Section 4.4 reproduces every learned recommendation (Table 6); confirmation outcomes cannot distinguish these selectors at the tested budgets. The compute-only choices use the original ridge-10 fit. Its quarter- and half-budget choices are absent from the confirmation plan, so this evaluation cannot compare those decisions on held-out tasks.

The selected allocation model has log loss 0.7048, compared with 0.7067 for the best compute-only model. It does not beat the trivial constant-$p = 0.5$ baseline, which scores 0.6931 under the same mean natural-log binary cross-entropy; an intercept-only model fitted separately on each training fold scores 0.7170. The response-model probabilities should therefore not be read as calibrated held-out-task forecasts. The primary test evaluates a recommendation shared with simpler selectors, so it does not establish an additional benefit from model-based ranking.

C Held-Out Outcomes and Verification

C.1 Complete confirmation outcomes

This subsection first reports success for every held-out task and setting, then reports the four comparisons added after collection began. Of those four, two isolate the execution prefix, one isolates denoising, and one changes both. The held-out evaluation uses Goal and Object tasks 5–8, states 20–29, and rollout seeds 3000 and 3001. Its 80 task–state blocks each contain both seeds and all seven configurations.

Table 8: Confirmation successes by task and configuration. Each task row has 20 attempts: ten paired initial states, each with two rollout seeds. The total aggregates 160 attempts per configuration. The primary $(2, 2)$ setting exceeds $(2, 5)$ on seven tasks and ties on one.

	$h = 1$			$h = 2$ (primary pair)		$h = 5$	
Task	$d = 2$	$d = 5$	$d = 10$	$d = 2$	$d = 5$	$d = 2$	$d = 5$
Goal 5	14	15	14	16	14	18	16
Goal 6	18	17	17	18	15	19	18
Goal 7	20	20	20	19	18	18	18
Goal 8	19	18	18	18	14	18	15
Object 5	20	19	19	20	20	20	20
Object 6	19	18	19	20	17	18	18
Object 7	9	9	10	10	7	7	8
Object 8	18	20	18	18	17	20	20
Total (/160)	137	136	135	139	122	138	133

Table 9: Amended descriptive comparisons using all 160 attempts per setting. These contrasts were documented after collection began and are not originally prespecified or fully outcome-blind. Intervals are task-conditional, unadjusted, and do not test whether two settings preserve quality within a prespecified margin.

Setting A	Setting B	A−B (pp)	95% interval (pp)
$(2, 2)$	$(1, 10)$	$+2.5$	$[-3.8, +8.1]$
$(2, 2)$	$(5, 2)$	$+0.6$	$[-6.3, +6.9]$
$(1, 2)$	$(2, 2)$	-1.3	$[-7.5, +5.0]$
$(1, 2)$	$(1, 10)$	$+1.3$	$[-5.0, +7.5]$

The four amended comparisons are reported in their documented order, using 20,000 paired state-bootstrap resamples and analysis seed 20260913. Table 9 reports their success differences and uncertainty intervals, while Table 1 gives their all-attempt resource means. In particular, the $(2, 2)$ minus $(5, 2)$ expenditure difference is $+25.0$ TFLOPs per attempt, with descriptive interval $[+22.8, +27.5]$. This interval describes generated-chunk arithmetic, including the different trajectory lengths; it is not an elapsed-time comparison or evidence of equivalent quality.

C.2 Primary contrast robustness, expenditure, and runtime

The primary selected configuration $(2, 2)$ has a higher success count than $(2, 5)$ on seven tasks and ties on one (Table 8). There are 24 paired wins and seven losses. In a descriptive leave-one-task-out sensitivity analysis, the equally weighted difference over the remaining seven tasks stays positive, ranging from 9.3 to 12.1 percentage points. This post-outcome analysis does not extend inference to a new task population. The BenchStudio example's paired episode-level records retain all state and seed identities needed to reproduce the primary counts and comparison.

Table 10: Descriptive primary expenditure differences, selected minus comparator at $h = 2$, including all attempts. Intervals use the paired state bootstrap conditional on the eight tasks and were added after outcomes were observed. Timing intervals summarize sampling variation in the observed records; they do not account for possible runtime interference during monitoring gaps. Differences are computed from unrounded means, so they need not equal the subtraction of the displayed one-decimal means.

Mean per attempt	$d = 2$	$d = 5$	Difference	95% interval
Reference TFLOPs	41.1	48.7	-7.6	$[-10.6, -4.5]$
Observed rollout (s)	19.0	34.3	-15.2	$[-17.2, -13.3]$
Control steps	149.0	166.3	-17.3	$[-27.9, -6.5]$

Table 11: Runtime and evaluation conditions shared by all four confirmation execution phases. Module versions are the loaded versions reported at evaluation time. The frozen paper records retain source digests and distinguish conflicting installed distribution metadata from these loaded versions.

Setting	Value
Hardware	RTX 4060 Ti, 16 GB; driver 595.84
Runtime	Python 3.12.3; PyTorch 2.7.1+cu126; CUDA 12.6; NumPy 2.2.6; MuJoCo 3.8.1
Evaluation	Batch 1; synchronous simulation; nominal 20 Hz (50 ms control period)
Episode horizons	Spatial and Object: 280 control steps; Goal: 300
Precision controls	Autocast and TF32 disabled; deterministic algorithms enabled
Attention and reuse	Eager language/action attention with float32 scores; within-generation KV cache enabled
Compilation/warm-up	Model compilation disabled; 20 warm-up generations per condition
Evaluator provenance	Pinned source revision and retained source-tree digest

Runtime and environment. The retained provenance does not characterize numerical precision and attention execution for every model component. Table 11 therefore supports the stated model configuration, but not component- or kernel-level claims.

C.3 Measurement verification

The retained records support a complete internal reconciliation of planned identities, paired states, executed actions, and recorded outcomes. The checks in Table 12 constrain recording and execution errors; they do not constitute an independent outcome replay in a separate simulator.

Table 12: Internal measurement-verification checklist.

Check	Evidence
Canonical coverage	1,120 confirmation attempts and 171,805 actions; the primary subset remains 320 attempts and 50,451 actions.
Pairing and dispatch	Planned identities, paired initial states and observations, generation counts, denoising counts, and applied actions reconcile.
Excluded records	Two repeated completed attempts are checked and excluded; one interrupted ten-action record has no completed outcome and is excluded.
Outcome path	All 1,122 raw completed records and 172,243 steps reconcile; generated actions match simulator actions and recorded outcome fields agree.
Shared predicate	Reward and success both use LIBERO's task-goal evaluation, traced to BDDL predicates, so their agreement is an internal consistency check.
Native parity	One Spatial episode matches the upstream path in initial state, observation, 70-action sequence, length, and outcome.
Remaining boundary	Independent, version-matched replay and direct terminal-state inspection remain outside this audit.

C.4 Analysis provenance

The three refinement comparisons, identities, and primary analysis were frozen before confirmation outcomes. The initial phase ended after 799 completions and one interrupted attempt. After 114 attempts per primary setting, a deadline-driven order amendment prioritized primary completion, adding 92 attempts; continuation phases added 6 and 225 to finish the original schedule, not new cohorts. Settings, stopping rules, and inclusion remained fixed. Monitoring covers the initial phase and later continuation, but not the priority phase or start of continuation. Deduplication retained the earliest completed record under this precedence. The frozen analyses use seed 20260912. Amended contrasts and diagnostics use seed 20260913 and are descriptive because they were added after collection began. No response model was refitted on confirmation outcomes.

Analysis artifact. The public BenchStudio example contains compact episode-level outcomes and a verification script that reproduces the held-out totals, paired identities, and primary success difference. The retained study archive additionally contains the detailed action records, plans, calibrations, frozen predictions, runtime provenance, and analysis inputs used for this manuscript. Its cryptographic identities are recorded in the public example. Checkpoint weights, action tensors, images, and simulator assets are omitted.